CS342 · VISUAL PROGRAMMING

Chapter 1 OOP & Exceptions

Midterm summary

Muharib

What's inside

01 Classes and objects

02 Inheritance

03 Polymorphism

04 Casting

05 Packages, final and abstract

06 Interfaces

07 Exceptions: the idea and the types

08 throws, throw and try-catch

09 finally and rethrowing

10 Custom exceptions

11 How to solve Find Output

01 Classes and objects

A **class** describes the structure and behavior of a set of similar objects. A specific instance of a class is called an **object**.
An object has a unique identity, **state** (data fields with their current values) and **behaviors** (methods).

The class is the template and the object is a real copy of it. The state is the variable values and the behavior is the methods.

Class diagram compartment	Contains
Top	The class name
Middle	Attributes (member variables)
Bottom	Operations (member functions)

```
Student s;                  // declaration
s = new Student();          // creation: calls the constructor
s.name = "Ali";             // access a public field with the dot
s.setName("Ahmad");         // call a method
```

Slide example: a class with several constructors (same name, different parameters); this means the current object.

```
public class Holiday {
    private String name;
    private int day;
    private String month;
    public Holiday(String name, int day, String month) {
        this.name = name; this.day = day; this.month = month;
    }
    public Holiday(String name, String month) {
        this.name = name; this.month = month; this.day = 7;
    }
    public Holiday(String name) {
        this.name = name; this.day = 23; this.month = "September";
    }
    public boolean sameMonth(Holiday h) {
        return this.month.equals(h.month);
    }
    double avgDay(Holiday[] hol) {
        int sum = 0;
        for (int i = 0; i < hol.length; i++) sum += hol[i].day;
        return ((double) sum / hol.length);
    }
}
```

```
hd[0] = new Holiday("Vacation1", 6, "November");   // day 6
hd[1] = new Holiday("Vacation2", "November");      // day 7
hd[2] = new Holiday("Vacation3");                  // day 23, September
hd[0].sameMonth(hd[2])   →  November vs September  →  not equal
hd[2].avgDay(hd)         →  (6 + 7 + 23) ÷ 3 = 12.0
```

02 Inheritance

```
public class Lawyer extends Employee {
    public Lawyer(int years) {
        super(years);                // calls the Employee constructor
    }
    @Override
    public double getSalary() {      // override: a new version in the subclass
        double baseSalary = super.getSalary();   // calls the parent method
        return baseSalary + 5000.00;
    }
}
```

Keyword	What it does
extends	The class inherits from another class (the superclass)
override	Replace a superclass method by writing a new version in the subclass
@Override	Tells Java the method exists in the parent, so Java checks the override is correct
super.method()	Calls the parent method
super(params)	Calls the parent constructor

The class **Object** forms the root of the overall inheritance tree of all Java classes. Every class is implicitly a subclass of Object. It defines methods such as **toString()**, which returns a text representation of the object, usually so that it can be printed.

So System.out.println(obj) prints whatever toString() returns. If you write it in your class, your version is used.

instanceof asks whether a variable refers to an object of a given type; it returns true or false.

```
String s = "hello";   Point p = new Point();
s instanceof Point    →  false        p instanceof String  →  false
s instanceof String   →  true         p instanceof Object  →  true
p instanceof Point    →  true         s instanceof Object  →  true
null instanceof String →  false       null instanceof Object →  false
```

03 Polymorphism

Polymorphism: ability for the same code to be used with different types of objects and behave differently with each.

The same line, but each object runs its own version. It is built on overriding.

```
Animal someanimal;
someanimal = somecat;   someanimal.display();   // display() of Cat
someanimal = somebird;  someanimal.display();   // display() of Bird
for (int j = 0; j < 200; j++)
    someanimal[j].display();   // no need to know if it is a Cat or a Bird
```

The rule: Java picks the method by the real object type (the one after new), not by the variable type.

04 Casting

	Upcasting	Downcasting
Direction	Dog → Animal	Animal → Dog
Syntax	No cast operator (implicit)	Needs an explicit cast (Dog)
Risk	Always safe	May cause a run-time error

Objects once created always know their type. The object itself is NOT changed or converted: casting is telling the compiler to ignore the "type mismatch". You need to cast downwards to use methods defined only in the derived class.

```
Animal a = new Cat(...);
a.motion();              // syntax error: motion() is not a method of Animal
((Cat) a).motion();      // OK
Cat c = a;               // syntax error: can't assign a superclass to a subclass
Cat c = (Cat) a;         // OK
Animal b = new Bird(...);
Cat c2 = (Cat) b;        // compiles, but RUN-TIME ERROR (ClassCastException)
if (b instanceof Cat)    // check first to stay safe
    ((Cat) b).motion();
```

This is exactly the Find Output question in midterm 471. Ask two questions on every line:

1. At compile time: does the method exist in the variable type (or the cast type)? If not, it is a syntax error.
2. At run time: is the real object of the cast type or a subclass of it? If not, a ClassCastException stops the program at that line.

05 Packages, final and abstract

A **package** name is defined by using the keyword package as the first instruction. To use a class from another package: **import**.

```
package it.infn.ts;      // first line of the file
import it.infn.ts.Book;  // use Book from another package
```

Modifier	On a class	On a method
final	Does not allow subclassing	Cannot be redefined in a subclass
abstract	Cannot be instantiated	Has no body and must be redefined in a subclass

abstract is roughly the opposite of final: a general design that the child classes complete.

06 Interfaces

An **interface** describes *what* classes should do, without specifying *how* they should do it.
• all fields are static and final
• all methods have no body and are public
• no instances can be created from interfaces
• a class can implement more than one interface

```
interface NiceBehavior {
    public String getName();
    public void sayGoodBye();
}
class IOSerialBoard2 implements IOBoardInterface, NiceBehavior {
    ...   // must implement ALL the methods of BOTH interfaces
}
```

The slide question "Which methods should it implement?": all the methods of both interfaces.

	abstract class	interface
Keyword	**extends**	**implements**
How many	Only one	More than one
Methods	May have a body or be abstract	All without a body, all public
new	Not allowed	Not allowed

07 Exceptions: the idea and the types

When a program runs into a runtime error, the program terminates abnormally. Exception handling lets the program continue to run or terminate gracefully. It enables a method to throw an exception to its caller.

```
Object
 └── Throwable
      ├── Error  (LinkageError, VirtualMachineError ...)        → unchecked
      └── Exception (IOException, ClassNotFoundException ...)    → checked
           └── RuntimeException                                   → unchecked
                (ArithmeticException, NullPointerException,
                 IndexOutOfBoundsException, IllegalArgumentException ...)
```

Type	What it is
Error	System errors thrown by the JVM; rare, little you can do beyond notifying the user and terminating
Exception	Errors caused by your program and external circumstances; they can be caught and handled
RuntimeException	Programming errors: bad casting, out-of-bounds array access, numeric errors

Unchecked exceptions: RuntimeException, Error and their subclasses. Java does not force you to catch them.
Checked exceptions: all the others. The compiler forces the programmer to check and deal with them.

Exception	When it is thrown
NumberFormatException	Integer.parseInt("x"): the text is not a number, is empty, or has a decimal point like "2.3"
ArithmeticException	Integer division by zero
NullPointerException	Using a reference before an object is assigned to it
IndexOutOfBoundsException	An index outside the bounds of the array
ClassCastException	A downcast to the wrong type
InputMismatchException	Scanner reads a value of the wrong type

08 throws, throw and try-catch

```
method1() {                          method2() throws Exception {   // declare
    try {                                if (an error occurs) {
        invoke method2;                      throw new Exception();     // throw
    }                                    }
    catch (Exception ex) {           }
        Process exception;           // catch
    }
}
```

Step	Syntax	Meaning
Declare	**throws**	In the method header: this method may throw this type
Throw	**throw new X()**	Create and throw the exception when the error is detected
Catch	**try {} catch (X e) {}**	Catch and handle it

Note: throws (with s) goes in the method header; throw (without s) goes in the code.

```
public void setRadius(double newRadius) throws IllegalArgumentException {
    if (newRadius >= 0)
        radius = newRadius;
    else
        throw new IllegalArgumentException("Radius cannot be negative");
}
```

Catch or declare: if p2 may throw a checked exception (e.g. IOException), p1 must either call it inside try-catch (a) or declare throws IOException (b).

```
(a) void p1() {                              (b) void p1() throws IOException {
        try { p2(); }                                p2();
        catch (IOException ex) { ... }           }
    }
```

Catch order: Java tries the catch blocks top to bottom, enters the first one that matches and skips the rest. After it, execution continues after the whole try statement, not from the line that failed.

09 finally and rethrowing

```
try {
    statements;
}
catch (TheException ex) {
    handling ex;
    throw ex;          // rethrow: control goes to the caller
}
finally {
    finalStatements;   // ALWAYS executed
}
Next statement;
```

Case	What runs
No exception	try fully → finally → Next statement
An exception with a matching catch	try up to the error → catch → finally → Next statement
The catch rethrows	catch → finally → back to the caller; Next statement is skipped
An exception with no matching catch	finally → back to the caller

A traced example with its output:

```
static void m(int x) throws Exception {
    try {
        System.out.println("A");
        if (x == 1) throw new ArithmeticException();
        if (x == 2) throw new Exception("boom");
        System.out.println("B");
    }
    catch (ArithmeticException e) { System.out.println("C"); }
    finally { System.out.println("D"); }
    System.out.println("E");
}
// main calls m(0), m(1), m(2), each inside try/catch printing "main: " + e.getMessage()
```

```
m(0):  A  B  D  E
m(1):  A  C  D  E
m(2):  A  D  main: boom
```

10 Custom exceptions

Use the exception classes in the API whenever possible. Define custom exception classes if the predefined classes are not sufficient, by extending Exception or a subclass of Exception.

```
public class ExceptionRange extends Exception {
    public ExceptionRange(String s) {
        super(s);                       // the message, read later with getMessage()
    }
}
public class ExceptionEmpty extends Exception {
    public String toString() {
        return ("No note is valid");    // printed by System.out.println(er)
    }
}
```

You write	It prints
e.getMessage()	The text passed to super(s)
System.out.println(e)	Whatever toString() returns

When to use exceptions: for unexpected error conditions, not for simple expected situations. Exception handling requires more time and resources (a new exception object, rolling back the call stack, propagating the error).

```
try { System.out.println(refVar.toString()); }          // avoid
catch (NullPointerException ex) { System.out.println("refVar is null"); }
if (refVar != null) System.out.println(refVar.toString());   // better
else System.out.println("refVar is null");
```

11 How to solve Find Output

1. Start at main, go line by line, and write each println as soon as it runs.
2. Inside a loop, take each iteration alone. An exception stops that iteration; a catch inside the loop lets the loop continue.
3. Integer.parseInt: anything that is not a whole number throws NumberFormatException: letters, " ", or "2.3".
4. Casts: check the real object type (after new).
5. If an exception is never caught, the program stops at that line and nothing after it prints.